

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение высшего образования
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ТОМСКИЙ ПОЛИТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ»



Инженерная школа информационных технологий и робототехники

Отделение информационных технологий

09.04.01 «Информатика и вычислительная техника»

Отчет по лабораторной работе №5
«Реализация трансферного обучения»

МЕТОДЫ РАСПОЗНАВАНИЯ ОБРАЗОВ

Исполнитель:

студент группы

8BM22

Ямкин Н.Н.

Руководитель:

Доцент (ОИТ, ИШИТР)

Друки А.А.

Томск – 2023

Содержание

1. Цель работы	3
2. Задачи	3
3. Ход работы	3
4. Заключение.....	6

1. Цель работы

Реализовать трансферное обучение. Импортировать выборочно отдельные слои предобученной нейросети для классификации объектов на изображениях. Создать нейронную сеть из импортированных слоев и своих оригинальных слоев.

2. Задачи

1. Загрузить и обработать датасет;
2. Создать нейронную сеть из импортированных слоев и своих оригинальных слоев;
3. Обучить и протестировать созданную нейронную сеть.

3. Ход работы

Создадим новый блокнот в Google Colab и импортируем необходимые для работы с изображениями библиотеки:

```
from keras.applications.vgg16 import VGG16
from keras.models import Model
from keras.layers import Input, Flatten, Dense
from keras.optimizers import Adam
from keras.layers import Conv2D, Flatten, Dense
from keras.models import Model
import numpy as np
from keras.datasets import cifar10
from keras.utils import to_categorical
from keras.layers import Dense, Dropout, Flatten
from keras.optimizers import SGD
from keras.preprocessing.image import ImageDataGenerator
from keras import regularizers
```

Загрузим и обработаем исходный набор данных.

rotation_range задает диапазон углов случайного поворота.

width_shift_range и **height_shift_range** задают диапазон случайных сдвигов по горизонтали и вертикали.

horizontal_flip указывает, следует ли случайным образом переворачивать изображение по горизонтали.

datagen.fit(X_train) адаптирует изображения из обучающего набора, в результате чего вычисляется некоторая необходимая статистическая информация (например, среднее значение, стандартное отклонение и т.д.) для последующих операций по улучшению данных. В процессе обучения можно использовать метод **datagen.flow()** для пакетного получения улучшенных данных.

```
# Загрузка набора данных CIFAR-10
(X_train, y_train), (X_test, y_test) = cifar10.load_data()

# Data preprocessing
X_train = X_train / 255.0
X_test = X_test / 255.0
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# Расширение данных с помощью ImageDataGenerator
datagen = ImageDataGenerator(
    rotation_range=15,
    width_shift_range=0.1,
    height_shift_range=0.1,
    horizontal_flip=True
)
datagen.fit(X_train)
```

Инициализируем модель VGG16.

```
vgg_model = VGG16(weights='imagenet', include_top=False, input_shape=(32,
32, 3))
```

Загружается предварительно обученная модель VGG16, размер входного изображения задается равным 32x32 пикселя с 3 каналами (RGB-изображение).

Определим архитектуру нейронной сети.

```
#написать код - архитектуру нейронной сети
x = vgg_model.get_layer('block1_conv1').output
x = vgg_model.get_layer('block1_conv2')(x)
x = vgg_model.get_layer('block1_pool')(x)

x = Conv2D(128, 3, activation='relu', padding='same')(x)
x = Conv2D(128, 3, activation='relu', padding='same')(x)
x = vgg_model.get_layer('block2_pool')(x)

x = Conv2D(256, 3, activation='relu', padding='same')(x)
x = Conv2D(256, 3, activation='relu', padding='same')(x)
x = Conv2D(256, 3, activation='relu', padding='same')(x)
x = vgg_model.get_layer('block3_pool')(x)

x = Conv2D(512, 3, activation='relu', padding='same')(x)
x = Conv2D(512, 3, activation='relu', padding='same')(x)
x = Conv2D(512, 3, activation='relu', padding='same')(x)
x = vgg_model.get_layer('block4_pool')(x)

x = Conv2D(512, 3, activation='relu', padding='same')(x)
x = Conv2D(512, 3, activation='relu', padding='same')(x)
x = Conv2D(512, 3, activation='relu', padding='same')(x)
x = vgg_model.get_layer('block5_pool')(x)

# Добавление полностью подключенного слоя и слоя с Dropout
x = Flatten()(x)
x = Dense(4096, activation='relu')(x)
x = Dropout(0.5)(x)
x = Dense(4096, activation='relu')(x)
x = Dropout(0.5)(x)

# задача классификации состоит из 10 классов
x = Dense(units=10, activation='softmax')(x)
```

Обучим и протестируем нейронную сеть.

```
# Создание новой модели
model = Model(inputs=vgg_model.input, outputs=x)

# Использование оптимизатора SGD
model.compile(loss='categorical_crossentropy', metrics=['accuracy'],
optimizer=SGD())

# Заморозьте первые 10 слоев
for layer in model.layers[:10]:
    layer.trainable = False

# Вывод имени каждого слоя и информации о том, является ли он обучаемым
for layer in model.layers:
    print(layer.name, layer.trainable)
```

```
# Модели обучения
model.fit(datagen.flow(X_train, y_train, batch_size=20), epochs=50,
validation_data=(X_test, y_test))
model.evaluate(X_test, y_test)
```

model = Model(inputs=vgg_model.input, outputs=x) создает новую модель, где `vgg_model.input` представляет собой входные данные для VGG16, а `x` - выходной результат, который вы определили в предыдущем коде.

model.compile(loss='categorical_crossentropy', metrics=['accuracy'], optimizer=Adam()) компилирует модель, указывая функцию потерь (`categorical_crossentropy`), метрики (`accuracy`) и оптимизатор (`SGD()`).

```
Epoch 46/50
2500/2500 [=====] - 79s 32ms/step - loss: 0.5321 - accuracy: 0.8152 - val_loss: 0.6375 - val_accuracy: 0.7915
Epoch 47/50
2500/2500 [=====] - 79s 32ms/step - loss: 0.5211 - accuracy: 0.8176 - val_loss: 0.6004 - val_accuracy: 0.8063
Epoch 48/50
2500/2500 [=====] - 81s 33ms/step - loss: 0.5141 - accuracy: 0.8208 - val_loss: 0.5906 - val_accuracy: 0.8058
Epoch 49/50
2500/2500 [=====] - 79s 32ms/step - loss: 0.5053 - accuracy: 0.8255 - val_loss: 0.5751 - val_accuracy: 0.8095
Epoch 50/50
2500/2500 [=====] - 80s 32ms/step - loss: 0.4972 - accuracy: 0.8263 - val_loss: 0.5652 - val_accuracy: 0.8111
<keras.src.callbacks.history at 0x783b60b330d0>
```

Рисунок 1 – Результаты обучения модели

4. Заключение

В данном эксперименте в качестве базовой модели была выбрана предварительно обученная нейронная сеть (VGG16). Затем были импортированы некоторые из слоев модели и добавлены несколько собственных слоев для классификации объектов на изображении.

Итоговая точность обучающего набора: 82,63%. Итоговая точность валидационного набора: 81,11%.